

bone__project,__seer,__v3

April 28, 2026

1 SEER tabular data

Objective: Use tabular data from SEER, and see if, when hybridized with malignance probabilities from Shah and Andy's NNs, whether or not classification improves

```
[2]: import pandas as pd
import numpy as np

file_path = "bones_seer.csv"
seer_df = pd.read_csv(file_path)

if 'Site recode ICD-0-3/WHO 2008' in seer_df.columns:
    seer_df = seer_df.drop(columns=['Site recode ICD-0-3/WHO 2008'])

print(seer_df.shape)
seer_df.head()
```

(32952, 15)

```
[2]: Age recode with <1 year olds and 90+      Sex  Year of diagnosis PRCDA 2020 \
0          50-54 years  Female          2018  Not PRCDA
1          45-49 years  Female          2001  Not PRCDA
2          55-59 years   Male          2002  Not PRCDA
3          80-84 years   Male          2020  Not PRCDA
4          60-64 years   Male          2008  Not PRCDA
```

```
      Race recode (W, B, AI, API) Origin recode NHIA (Hispanic, Non-Hisp) \
0          Black          Non-Spanish-Hispanic-Latino
1          Black          Non-Spanish-Hispanic-Latino
2          White          Non-Spanish-Hispanic-Latino
3          White          Non-Spanish-Hispanic-Latino
4          White          Non-Spanish-Hispanic-Latino
```

```
      Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic) \
0          Non-Hispanic Black
1          Non-Hispanic Black
2          Non-Hispanic White
3          Non-Hispanic White
```

4

Non-Hispanic White

	Tumor Size Summary (2016+)	First malignant primary indicator \
0	290	No
1	Blank(s)	Yes
2	Blank(s)	No
3	043	No
4	Blank(s)	No

	Primary Site - labeled \
0	C40.0-Long bones: upper limb, scapula, and ass...
1	C41.0-Bones of skull and face and associated j...
2	C41.9-Bone, NOS
3	C41.3-Rib, sternum, clavicle and associated jo...
4	C41.3-Rib, sternum, clavicle and associated jo...

	Laterality	ICD-O-3 Hist/behav \
0	Right - origin of primary	9220/3: Chondrosarcoma, NOS
1	Not a paired site	9370/3: Chordoma, NOS
2	Not a paired site	9260/3: Ewing sarcoma
3	Left - origin of primary	9220/3: Chondrosarcoma, NOS
4	Left - origin of primary	9220/3: Chondrosarcoma, NOS

	Grade Clinical (2018+)	Grade Pathological (2018+) \
0	2	2
1	Blank(s)	Blank(s)
2	Blank(s)	Blank(s)
3	1	1
4	Blank(s)	Blank(s)

	Combined Summary Stage with Expanded Regional Codes (2004+)
0	Regional by direct extension only
1	Blank(s)
2	Blank(s)
3	Localized only
4	Regional by direct extension only

2 cleaning

2.1 1. age recoding

```
[3]: # 'Age recode with <1 year olds and 90+' comes in as strings like "50-54 years".
# Pull out the lower bound of each bin as a numeric feature.
seer_df['Age recode with <1 year olds and 90+'] = (
    seer_df['Age recode with <1 year olds and 90+']
    .astype(str)
    .str.extract(r'(\d+)')
```

```
.astype(float)
)
```

2.2 2. NaN handling

```
[4]: # Replace SEER's filler values with real NaNs so we can see the actual
      ↪missingness
seer_df = seer_df.replace(['Blank(s)', 'Unknown', 'unknown', 'None'], np.nan)
seer_df = seer_df.replace([999, 9999, '999', '9999'], np.nan)

print(f"Rows: {len(seer_df)}")
print("\nMissing values per column:")
print(seer_df.isna().sum())
```

Rows: 32952

Missing values per column:

Age recode with <1 year olds and 90+	0
Sex	0
Year of diagnosis	0
PRCDA 2020	0
Race recode (W, B, AI, API)	379
Origin recode NHIA (Hispanic, Non-Hisp)	0
Race and origin recode (NHW, NHB, NHAIAN, NHAPIO, Hispanic)	0
Tumor Size Summary (2016+)	24477
First malignant primary indicator	0
Primary Site - labeled	0
Laterality	0
ICD-O-3 Hist/behav	0
Grade Clinical (2018+)	24770
Grade Pathological (2018+)	24770
Combined Summary Stage with Expanded Regional Codes (2004+)	5090
dtype: int64	

2.3 3. column wrangling for target var

```
[5]: #make col
seer_df['Is_Malignant'] = seer_df['ICD-O-3 Hist/behav'].str.contains('/3:').
      ↪astype(int)
#delete remnant col
seer_df = seer_df.drop(columns=['ICD-O-3 Hist/behav'])
print(seer_df['Is_Malignant'].value_counts())
```

Is_Malignant

1 32902

0 50

Name: count, dtype: int64

2.4 4. dealing w cols w big NAs, rows small NAs

```
[6]: #so we're gonna rid of the columns w heavyt NAs and then get rid of the rows
      ↪for things with small NAs
      #delete the high NA cols
      columns_to_drop = [
          'Grade Clinical (2018+)',
          'Grade Pathological (2018+)',
          'Tumor Size Summary (2016+)']

      seer_df = seer_df.drop(columns=columns_to_drop)

      #make state NAs unknown
      stage_col = 'Combined Summary Stage with Expanded Regional Codes (2004+)'
      seer_df[stage_col] = seer_df[stage_col].fillna('Unknown')

      #delete the ones missing race
      seer_df = seer_df.dropna()

      #final ct
      print(f"Final Patient Count: {len(seer_df)}")
```

Final Patient Count: 32573

2.5 5. one-hot encoding

```
[7]: # We drop the granular Race recode columns because the combined
      # Race-and-origin recode already captures the same information.
      categorical_cols = [
          'Sex',
          'Race recode (W, B, AI, API)',
          'Primary Site - labeled',
          'Laterality',
          'Combined Summary Stage with Expanded Regional Codes (2004+)',
          'PRCDA 2020',
          'Origin recode NHIA (Hispanic, Non-Hisp)',
          'Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)',
          'First malignant primary indicator',
      ]

      seer_df = pd.get_dummies(seer_df, columns=categorical_cols, drop_first=True,
                              ↪dtype=int)

      # Drop redundant Race recode columns (info preserved in Race and origin recode)
      redundant_race_cols = [c for c in seer_df.columns if c.startswith('Race recode_
                              ↪(W, B, AI, API)')]
      seer_df = seer_df.drop(columns=redundant_race_cols)

      remaining_text = seer_df.select_dtypes(include=['object']).columns.tolist()
```

```
print(f"Shape after encoding: {seer_df.shape}")
print(f"Remaining text columns (should just be Age): {remaining_text}")
```

Shape after encoding: (32573, 35)

Remaining text columns (should just be Age): []

2.6 6. restrict sites to whats in MURA (hands etc)

```
[8]: # MURA only covers upper limb / shoulder studies. Keep only SEER patients whose
# tumors fall in those sites so the two datasets describe comparable anatomy.
non_upper_extremity_sites = [
    'Primary Site - labeled_C40.2-Long bones of lower limb and associated_
    ↪joints',
    'Primary Site - labeled_C40.3-Short bones of lower limb and associated_
    ↪joints',
    'Primary Site - labeled_C40.8-Overlap of bones, joints, and art. cartilage_
    ↪of limbs',
    'Primary Site - labeled_C40.9-Bone of limb, NOS',
    'Primary Site - labeled_C41.0-Bones of skull and face and associated_
    ↪joints',
    'Primary Site - labeled_C41.1-Mandible',
    'Primary Site - labeled_C41.2-Vertebral column',
    'Primary Site - labeled_C41.3-Rib, sternum, clavicle and associated joints',
    'Primary Site - labeled_C41.4-Pelvic bones, sacrum, coccyx and associated_
    ↪joints',
    'Primary Site - labeled_C41.8-Overlap bones, joints, and art. cartilage',
]
mask = (seer_df[non_upper_extremity_sites] == 0).all(axis=1)
seer_df = seer_df[mask].copy()

print(f"Upper-extremity patients: {len(seer_df)}")
print(seer_df['Is_Malignant'].value_counts())
```

Upper-extremity patients: 5448

Is_Malignant

1 5440

0 8

Name: count, dtype: int64

2.7 7. Drop leakage columns, freeze the canonical clean dataset

```
[9]: # Drop target/admin leakage:
#     - Stage encodes outcome severity that wouldn't be known at the moment we'd
#       want to predict malignancy
#     - PRCDA is administrative
#     - Year of diagnosis encodes when reporting standards changed, not biology
leakage_cols = [c for c in seer_df.columns if 'Stage' in c or 'PRCDA' in c]
```

```

leakage_cols.append('Year of diagnosis')
seer_df = seer_df.drop(columns=leakage_cols)

# Freeze the cleaned dataset under a new name. Do not reassign clean_df anywhere
# downstream - every modeling section should read from it and write to its own
↳ variable.
clean_df = seer_df.copy()

print(f"Clean dataset: {len(clean_df)} rows, "
      f"{clean_df['Is_Malignant'].sum()} malignant, "
      f"{(clean_df['Is_Malignant']==0).sum()} benign")
print(f"Features: {clean_df.shape[1] - 1}")

```

Clean dataset: 5448 rows, 5440 malignant, 8 benign
Features: 25

3 sampling models begin

3.1 round 1: oversampling

```

[10]: from sklearn.utils import resample
      from sklearn.ensemble import RandomForestClassifier
      from sklearn.metrics import classification_report, accuracy_score
      from sklearn.model_selection import train_test_split

[11]: print("--- FAILURE 1: STANDARD OVERSAMPLING ---")
      X_fail1 = clean_df.drop(columns=['Is_Malignant'])
      y_fail1 = clean_df['Is_Malignant']

      X_train_f1, X_test_f1, y_train_f1, y_test_f1 = train_test_split(
          X_fail1, y_fail1, test_size=0.2, random_state=4337, stratify=y_fail1)

      # Recombine to oversample
      train_df_f1 = pd.concat([X_train_f1, y_train_f1], axis=1)
      df_majority_f1 = train_df_f1[train_df_f1['Is_Malignant'] == 1]
      df_minority_f1 = train_df_f1[train_df_f1['Is_Malignant'] == 0]

      df_minority_upsampled_f1 = resample(
          df_minority_f1, replace=True,
          n_samples=len(df_majority_f1), random_state=4337)
      oversampled_train_df = pd.concat([df_majority_f1, df_minority_upsampled_f1])

      rf_fail1 = RandomForestClassifier(n_estimators=100, random_state=4337)
      rf_fail1.fit(oversampled_train_df.drop(columns=['Is_Malignant']),
                  oversampled_train_df['Is_Malignant'])
      preds_fail1 = rf_fail1.predict(X_test_f1)

```

```
print(f"Overall Accuracy: {accuracy_score(y_test_f1, preds_fail1) * 100:.
↪2f}%\n")
print(classification_report(y_test_f1, preds_fail1))
```

--- FAILURE 1: STANDARD OVERSAMPLING ---

Overall Accuracy: 97.71%

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	1.00	0.98	0.99	1088
accuracy			0.98	1090
macro avg	0.50	0.49	0.49	1090
weighted avg	1.00	0.98	0.99	1090

Class 0 recall is 0. Bad. Just predicts everything as malignant, cuz the sampling is so imbalanced.

3.2 round 2: smote

```
[12]: from imblearn.over_sampling import SMOTE

print("--- FAILURE 2: SMOTE OVERSAMPLING ---")
# Reuses X_train_f1, X_test_f1, y_train_f1, y_test_f1 from the oversampling cell

# Apply SMOTE only to the training data.
# k_neighbors=5 is the default but worth keeping explicit - with only 8 benigns
# in the full dataset (6 in train), SMOTE is interpolating between a tiny
# handful of points, so every neighbor counts.
smote_fail = SMOTE(random_state=4337, k_neighbors=5)
X_train_smote_f2, y_train_smote_f2 = smote_fail.fit_resample(X_train_f1, ↪
↪y_train_f1)

# Train and test
rf_fail2 = RandomForestClassifier(n_estimators=100, random_state=4337)
rf_fail2.fit(X_train_smote_f2, y_train_smote_f2)
preds_fail2 = rf_fail2.predict(X_test_f1)

print(f"Overall Accuracy: {accuracy_score(y_test_f1, preds_fail2) * 100:.
↪2f}%\n")
print(classification_report(y_test_f1, preds_fail2, zero_division=0))

print("Class distribution after SMOTE:")
print(y_train_smote_f2.value_counts())
```

--- FAILURE 2: SMOTE OVERSAMPLING ---

Overall Accuracy: 99.82%

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	1.00	1.00	1.00	1088
accuracy			1.00	1090
macro avg	0.50	0.50	0.50	1090
weighted avg	1.00	1.00	1.00	1090

Class distribution after SMOTE:

Is_Malignant

1 4352

0 4352

Name: count, dtype: int64

AGAIN: Class 0 recall is 0. Bad. Just predicts everything as malignant, cuz the sampling is so imbalanced.

3.3 round 3: monte carlo

Naive MC. Bootstrap real rows, flip label to 0, add noise to age, scramble sex. Doesn't work

```
[13]: mc_v1_df = clean_df.copy()
simulated_df = mc_v1_df.sample(n=5000, replace=True, random_state=4337).copy()
simulated_df['Is_Malignant'] = 0

# Add noise to age and scramble sex so synthetic rows aren't carbon copies of
↳ donors
age_col = 'Age recode with <1 year olds and 90+'
rng = np.random.default_rng(4337)
simulated_df[age_col] = (simulated_df[age_col] + rng.integers(-10, 11,
↳ size=5000)).clip(1, 90)
simulated_df['Sex_Male'] = rng.permutation(simulated_df['Sex_Male'].values)

mc_v1_df = pd.concat([mc_v1_df, simulated_df], ignore_index=True)
print(f"Round 3 dataset: {len(mc_v1_df)} rows")
print(mc_v1_df['Is_Malignant'].value_counts())
```

Round 3 dataset: 10448 rows

Is_Malignant

1 5440

0 5008

Name: count, dtype: int64

```
[14]: # Train / test split
X = mc_v1_df.drop(columns=['Is_Malignant'])
y = mc_v1_df['Is_Malignant']
X_train_mc1, X_test_mc1, y_train_mc1, y_test_mc1 = train_test_split(
```



```

X, y, test_size=0.2, random_state=4337, stratify=y)

# Train baseline RF
rf_mc1 = RandomForestClassifier(n_estimators=100, random_state=4337)
rf_mc1.fit(X_train_mc1, y_train_mc1)
preds_default = rf_mc1.predict(X_test_mc1)

print("--- Default threshold (0.5) ---")
print(f"Accuracy: {accuracy_score(y_test_mc1, preds_default)*100:.2f}%")
print(classification_report(y_test_mc1, preds_default, zero_division=0))

# Lower threshold to favor recall on malignant
THRESHOLD = 0.20
probs = rf_mc1.predict_proba(X_test_mc1)[: , 1]
preds_paranoid = (probs >= THRESHOLD).astype(int)

print(f"\n--- Threshold = {THRESHOLD} ---")
print(f"Accuracy: {accuracy_score(y_test_mc1, preds_paranoid)*100:.2f}%")
print(classification_report(y_test_mc1, preds_paranoid, zero_division=0))

```

--- Default threshold (0.5) ---

Accuracy: 78.90%

	precision	recall	f1-score	support
0	0.85	0.68	0.76	1002
1	0.75	0.89	0.81	1088
accuracy			0.79	2090
macro avg	0.80	0.78	0.79	2090
weighted avg	0.80	0.79	0.79	2090

--- Threshold = 0.2 ---

Accuracy: 75.65%

	precision	recall	f1-score	support
0	0.93	0.53	0.68	1002
1	0.69	0.96	0.80	1088
accuracy			0.76	2090
macro avg	0.81	0.75	0.74	2090
weighted avg	0.80	0.76	0.74	2090

```

[15]: importances = (
      pd.Series(rf_mc1.feature_importances_, index=X_train_mc1.columns)
      .sort_values(ascending=False)

```

```

        .head(15)
    )
    print("Top 15 features (round 3):")
    print(importances.to_string())

```

```

Top 15 features (round 3):
Age recode with <1 year olds and 90+
0.944129
Sex_Male
0.010873
First malignant primary indicator_Yes
0.008164
Primary Site - labeled_C40.1-Short bones of upper limb and associated joints
0.006861
Primary Site - labeled_C41.9-Bone, NOS
0.005198
Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)_Non-Hispanic White
0.003458
Laterality_Left - origin of primary
0.003425
Laterality_Right - origin of primary
0.003332
Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)_Non-Hispanic Black
0.002863
Origin recode NHIA (Hispanic, Non-Hisp)_Spanish-Hispanic-Latino
0.002827
Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)_Non-Hispanic Asian or
Pacific Islander      0.002669
Laterality_Not a paired site
0.002160
Laterality_Paired site, but no information concerning laterality
0.002008
Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)_Non-Hispanic American
Indian/Alaska Native  0.001266
Laterality_Only one side - side unspecified
0.000768

```

At first, the model looks good (.96 recall!) but upon looking at feature importance it all falls apart. Age at .94 importance isn't a real model. So, we need to try again

3.4 round 4: improved monte carlo

3.4.1 4.1: leakage check

```

[16]: #any feature whose mean differs sharply between classes is a candidate shortcut
      print("Class distribution:")
      print(clean_df['Is_Malignant'].value_counts(), "\n")

```

```
leakage_candidates = [c for c in clean_df.columns if 'First malignant' in c]
for col in leakage_candidates:
    means = clean_df.groupby('Is_Malignant')[col].mean()
    print(f"{col}: {means.to_dict()}")
```

Class distribution:

Is_Malignant

1 5440

0 8

Name: count, dtype: int64

First malignant primary indicator_Yes: {0: 1.0, 1: 0.8834558823529411}

We found 'First malignant primary indicator_Yes' is ~0.88 in BOTH classes, so it's NOT leakage. Ergo, we keep it

```
[17]: malig_df = clean_df[clean_df['Is_Malignant'] == 1].copy().reset_index(drop=True)
      real_benign_df = clean_df[clean_df['Is_Malignant'] == 0].copy().
      ↪reset_index(drop=True)

      print(f"Malignant cases: {len(malig_df)}")
      print(f"Real benign cases: {len(real_benign_df)}")
```

Malignant cases: 5440

Real benign cases: 8

3.4.2 4.2: mc simulator

It mimicks benign tumor placement and other features as is observed in scientific literature

```
[18]: def simulate_benign_cases(malignant_df, n_synthetic, target_col='Is_Malignant',
                                random_state=4337):

    """
    Generate synthetic benign bone tumor cases from a donor pool of malignant_
    ↪rows.

    Design choices:
    - Demographics (age, sex, race, hispanic) copied from a random donor:
      benigns and malignants share roughly the same demographic distribution
      in real life, so this avoids manufacturing a demographic shortcut.
    - Anatomic site drawn from epidemiology-based weights, NOT copied from
      donor. ~62% of real cases sit in the reference category (dropped by
      get_dummies), so the simulator must produce REFERENCE rows at that rate
      or the model will learn "all-zero site columns = real = malignant".
    - Laterality copied from a SECOND independent donor - decorrelates from
      demographics so the model can't pair-match.
    - 'First malignant primary indicator' copied from donor: it's ~0.88 in
      both classes, so forcing it to 0 in v2 was leaking the label.
    """
```

```

rng = np.random.default_rng(random_state)
all_cols = malignant_df.columns.tolist()

site_cols      = [c for c in all_cols if c.startswith('Primary Site')]
laterality_cols = [c for c in all_cols if c.startswith('Laterality')]
race_cols      = [c for c in all_cols if 'Race and origin recode' in c]
hispanic_cols  = [c for c in all_cols if 'Origin recode NHIA' in c]
sex_cols       = [c for c in all_cols if c.startswith('Sex_')]
age_cols       = [c for c in all_cols if 'Age recode' in c]
first_malign_cols = [c for c in all_cols if 'First malignant primary_
↳indicator' in c]

benign_site_weights = {
    'REFERENCE': 5.0,
    'C40.1': 2.0, 'C40.2': 5.0, 'C40.3': 1.0, 'C40.8': 0.5, 'C40.9': 1.0,
    'C41.0': 1.5, 'C41.1': 0.8, 'C41.2': 1.0, 'C41.3': 0.8, 'C41.4': 1.5,
    'C41.8': 0.3, 'C41.9': 0.8,
}

site_choices = ['REFERENCE']
site_weights_list = [benign_site_weights['REFERENCE']]
for col in site_cols:
    for code, w in benign_site_weights.items():
        if code != 'REFERENCE' and code in col:
            site_choices.append(col)
            site_weights_list.append(w)
            break
site_weights_arr = np.array(site_weights_list) / sum(site_weights_list)

rows = []
for _ in range(n_synthetic):
    row = {c: 0 for c in all_cols}
    donor = malignant_df.iloc[rng.integers(0, len(malignant_df))]
    for col in age_cols + sex_cols + race_cols + hispanic_cols +
↳first_malign_cols:
        row[col] = donor[col]
    chosen = rng.choice(site_choices, p=site_weights_arr)
    if chosen != 'REFERENCE':
        row[chosen] = 1
    lat_donor = malignant_df.iloc[rng.integers(0, len(malignant_df))]
    for col in laterality_cols:
        row[col] = lat_donor[col]
    row[target_col] = 0
    rows.append(row)

return pd.DataFrame(rows, columns=all_cols)

```

3.4.3 4.3: build train/test and train rf, get results

```
[19]: from sklearn.metrics import confusion_matrix

# Split malignants 80/20. The 8 real benigns ALL go to test - we cannot afford
# to train on any of them, and 8 isn't enough to split anyway.
malig_train, malig_test = train_test_split(malig_df, test_size=0.2,
# random_state=4337)
real_benign_test = real_benign_df.copy()
feature_cols = [c for c in malig_df.columns if c != 'Is_Malignant']

# Synthetic benigns for training (donor pool = malig_train only, NEVER
# malig_test)
syn_train = simulate_benign_cases(malig_train, n_synthetic=len(malig_train),
# random_state=4337)
train_df = (
    pd.concat([malig_train, syn_train], ignore_index=True)
    .sample(frac=1, random_state=4337)
    .reset_index(drop=True)
)
X_train, y_train = train_df[feature_cols], train_df['Is_Malignant']

# Two test sets:
# Tier 1 - synthetic benigns (n=many, but tests synthesis-detection more than
# diagnosis)
# Tier 2 - the 8 real benigns (n=tiny, but real)
syn_test = simulate_benign_cases(malig_test, n_synthetic=len(malig_test),
# random_state=9999)
test_syn = pd.concat([malig_test, syn_test], ignore_index=True)
test_real = pd.concat([malig_test, real_benign_test], ignore_index=True)
X_test_syn, y_test_syn = test_syn[feature_cols], test_syn['Is_Malignant']
X_test_real, y_test_real = test_real[feature_cols], test_real['Is_Malignant']

# Train once
rf = RandomForestClassifier(n_estimators=100, random_state=4337,
# class_weight='balanced')
rf.fit(X_train, y_train)

print(f"Train: {len(X_train)} rows, balanced {y_train.value_counts().
# to_dict()}")
print(f"Tier 1 test (synthetic benigns): {len(y_test_syn)} rows")
print(f"Tier 2 test (real benigns): {len(y_test_real)} rows\n")

print("--- Default threshold (0.5) ---")
cm_syn = confusion_matrix(y_test_syn, rf.predict(X_test_syn), labels=[0, 1])
cm_real = confusion_matrix(y_test_real, rf.predict(X_test_real), labels=[0, 1])
```

```

print(f"Tier 1 (synthetic) - [[TN, FP], [FN, TP]]:\n{cm_syn}")
print(f"  benigns caught: {cm_syn[0,0]}/{cm_syn[0].sum()} | FN:␣
↪{cm_syn[1,0]}\n")

print(f"Tier 2 (real) - [[TN, FP], [FN, TP]]:\n{cm_real}")
print(f"  benigns caught: {cm_real[0,0]}/{cm_real[0].sum()} | FN:␣
↪{cm_real[1,0]}\n")

```

Train: 8704 rows, balanced {1: 4352, 0: 4352}

Tier 1 test (synthetic benigns): 2176 rows

Tier 2 test (real benigns): 1096 rows

--- Default threshold (0.5) ---

Tier 1 (synthetic) - [[TN, FP], [FN, TP]]:

[[850 238]

[66 1022]]

benigns caught: 850/1088 | FN: 66

Tier 2 (real) - [[TN, FP], [FN, TP]]:

[[0 8]

[66 1022]]

benigns caught: 0/8 | FN: 66

[20]: *# Top features in the round-4 RF. Should be much flatter than round 3*

```

importances = (
    pd.Series(rf.feature_importances_, index=feature_cols)
    .sort_values(ascending=False)
    .head(15)
)
print("Top 15 features (round 4):")
print(importances.to_string())

```

Top 15 features (round 4):

Primary Site - labeled_C40.2-Long bones of lower limb and associated joints

0.185192

Primary Site - labeled_C41.9-Bone, NOS

0.124660

Age recode with <1 year olds and 90+

0.117278

Laterality_Not a paired site

0.106312

Primary Site - labeled_C41.4-Pelvic bones, sacrum, coccyx and associated joints

0.058093

Primary Site - labeled_C41.0-Bones of skull and face and associated joints

0.056823

Primary Site - labeled_C41.2-Vertebral column

0.042282

Primary Site - labeled_C40.3-Short bones of lower limb and associated joints

0.042235
Primary Site - labeled_C40.9-Bone of limb, NOS
0.040291
Primary Site - labeled_C41.3-Rib, sternum, clavicle and associated joints
0.037520
Primary Site - labeled_C41.1-Mandible
0.033499
Laterality_Right - origin of primary
0.029316
Laterality_Left - origin of primary
0.029306
Primary Site - labeled_C40.1-Short bones of upper limb and associated joints
0.020386
Primary Site - labeled_C40.8-Overlap of bones, joints, and art. cartilage of limbs 0.020212

```
[21]: # Threshold sweep on Tier 1. Lower thresholds catch more malignants (fewer FN)
      ↪ at the cost of more FP. For "minimize FN", we want the lowest threshold
      ↪ where malig recall 0.99 without FP exploding.
probs_syn = rf.predict_proba(X_test_syn)[: , 1]

print(f"{'Threshold':<12}{'Malig Recall':<15}{'Malig Prec':<14}{'Benign Recall':<15}{'FN':<6}{'FP':<6}")
print("-" * 68)
for thresh in np.arange(0.10, 0.90, 0.05):
    preds = (probs_syn >= thresh).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test_syn, preds, labels=[0,1]).ravel()
    print(f"{'thresh':<12.2f}{'tp/(tp+fn)':<15.3f}{'tp/(tp+fp)':<14.3f}{'tn/(tn+fp)':<15.3f}{'fn:<6}{'fp:<6}"}")
```

Threshold	Malig Recall	Malig Prec	Benign Recall	FN	FP
0.10	0.994	0.776	0.713	7	312
0.15	0.990	0.781	0.722	11	302
0.20	0.987	0.782	0.724	14	300
0.25	0.983	0.787	0.733	19	290
0.30	0.976	0.790	0.740	26	283
0.35	0.966	0.794	0.749	37	273
0.40	0.958	0.796	0.755	46	267
0.45	0.952	0.802	0.766	52	255
0.50	0.939	0.811	0.781	66	238
0.55	0.919	0.812	0.787	88	232
0.60	0.903	0.821	0.803	106	214
0.65	0.865	0.823	0.814	147	202
0.70	0.841	0.830	0.827	173	188
0.75	0.788	0.832	0.841	231	173
0.80	0.654	0.857	0.891	376	119
0.85	0.505	0.862	0.919	539	88

We choose .20 for threshold based on the graph

```
[22]: THRESHOLD = 0.20

probs_real = rf.predict_proba(X_test_real)[: , 1]
preds_syn = (probs_syn >= THRESHOLD).astype(int)
preds_real = (probs_real >= THRESHOLD).astype(int)

print(f"=== TIER 1 @ threshold {THRESHOLD} ===")
print(f"Accuracy: {accuracy_score(y_test_syn, preds_syn)*100:.2f}%")
print(classification_report(y_test_syn, preds_syn,
                             target_names=['Benign', 'Malignant'],
                             ↪zero_division=0))
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(confusion_matrix(y_test_syn, preds_syn))

print(f"\n=== TIER 2 @ threshold {THRESHOLD} (n=8 benigns - limited validity) ↪
      ↪===")
print(f"Accuracy: {accuracy_score(y_test_real, preds_real)*100:.2f}%")
print(classification_report(y_test_real, preds_real,
                             target_names=['Benign', 'Malignant'],
                             ↪zero_division=0))
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(confusion_matrix(y_test_real, preds_real))
```

=== TIER 1 @ threshold 0.2 ===

Accuracy: 85.52%

	precision	recall	f1-score	support
Benign	0.98	0.72	0.83	1088
Malignant	0.78	0.99	0.87	1088
accuracy			0.86	2176
macro avg	0.88	0.86	0.85	2176
weighted avg	0.88	0.86	0.85	2176

Confusion matrix [[TN, FP], [FN, TP]]:

```
[[ 787  301]
 [  14 1074]]
```

=== TIER 2 @ threshold 0.2 (n=8 benigns - limited validity) ===

Accuracy: 97.99%

	precision	recall	f1-score	support
Benign	0.00	0.00	0.00	8
Malignant	0.99	0.99	0.99	1088
accuracy			0.98	1096

macro avg	0.50	0.49	0.49	1096
weighted avg	0.99	0.98	0.98	1096

Confusion matrix [[TN, FP], [FN, TP]]:

```
[[ 0  8]
 [ 14 1074]]
```

```
[23]: # Per-case real benign probabilities - the one thing actually informative on n=8
print("\n--- Real benign cases, sorted by P(malignant) ---")
real_benign_probs = rf.predict_proba(real_benign_test[feature_cols][:, 1])
rows = []
for i, p in enumerate(real_benign_probs):
    case = real_benign_test.iloc[i]
    site = next((c.replace('Primary Site - labeled_', '')[45]
                  for c in case.index if c.startswith('Primary Site') and
    case[c] == 1),
                'reference (no site flag)')
    sex = 'M' if case.get('Sex_Male', 0) == 1 else 'F'
    rows.append((p, i+1, case['Age recode with <1 year olds and 90+'], sex,
    site))
rows.sort()

print(f"{'P(malig)':<11}{'Case':<7}{'Age':<6}{'Sex':<5}{'Site'}")
for p, n, age, sex, site in rows:
    flag = '←FN risk' if p >= THRESHOLD else ''
    print(f"{'p':<11.3f}#{n:<6}{age:<6.0f}{sex:<5}{site} {flag}")
```

--- Real benign cases, sorted by P(malignant) ---

P(malig)	Case	Age	Sex	Site
0.741	#3	60	F	reference (no site flag) ←FN risk
0.763	#1	55	F	reference (no site flag) ←FN risk
0.776	#8	70	M	reference (no site flag) ←FN risk
0.798	#2	15	F	reference (no site flag) ←FN risk
0.883	#6	15	F	reference (no site flag) ←FN risk
0.889	#7	65	F	reference (no site flag) ←FN risk
0.899	#5	35	F	C41.9-Bone, NOS ←FN risk
1.000	#4	45	F	C41.9-Bone, NOS ←FN risk

P(malig)	Case	Age	Sex	Site
0.741	#3	60	F	reference (no site flag) ←FN risk
0.763	#1	55	F	reference (no site flag) ←FN risk
0.776	#8	70	M	reference (no site flag) ←FN risk
0.798	#2	15	F	reference (no site flag) ←FN risk
0.883	#6	15	F	reference (no site flag) ←FN risk
0.889	#7	65	F	reference (no site flag) ←FN risk
0.899	#5	35	F	C41.9-Bone, NOS ←FN risk
1.000	#4	45	F	C41.9-Bone, NOS ←FN risk

3.4.4 4.4. svm cascade idea on the real benigns

```
[24]: from sklearn.svm import SVC

# Two-stage cascade idea: RF at 0.15 casts a wide net (almost 0 FN).
# SVM then re-evaluates the things RF flagged, using its default 0.5 boundary
# as a strict skeptic. Hope is to drop FP without losing too many TP.

# Stage 2 trains on only the rows Stage 1 flagged in TRAINING (so the SVM sees
# the "borderline" subspace, not the easy cases)
train_flagged = rf.predict_proba(X_train)[: , 1] >= THRESHOLD
X_train_s2, y_train_s2 = X_train[train_flagged], y_train[train_flagged]
print(f"SVM training pool (RF-flagged train rows): {len(X_train_s2)}")
print(f"  class balance: {y_train_s2.value_counts().to_dict()}\n")

svm_refiner = SVC(probability=True, kernel='rbf', class_weight='balanced',
                  random_state=4337)
svm_refiner.fit(X_train_s2, y_train_s2)

def cascade_predict(X, rf_model, svm_model, threshold):
    """RF wide-net → SVM strict-skeptic on flagged rows only."""
    rf_probs = rf_model.predict_proba(X)[: , 1]
    flagged = rf_probs >= threshold
    preds = np.zeros(len(X), dtype=int) # everything not flagged stays benign
    if flagged.any():
        preds[flagged] = svm_model.predict(X[flagged])
    return preds

cascade_real = cascade_predict(X_test_real, rf, svm_refiner, THRESHOLD)

print(f"=== CASCADE (RF@{THRESHOLD} → SVM@0.5) on TIER 2 (real benigns) ===")
print(f"Accuracy: {accuracy_score(y_test_real, cascade_real)*100:.2f}%")
print(classification_report(y_test_real, cascade_real,
                           target_names=['Benign', 'Malignant'],
                           zero_division=0))
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(confusion_matrix(y_test_real, cascade_real))
```

```
SVM training pool (RF-flagged train rows): 5374
  class balance: {1: 4351, 0: 1023}
```

```
=== CASCADE (RF@0.2 → SVM@0.5) on TIER 2 (real benigns) ===
Accuracy: 26.55%
```

	precision	recall	f1-score	support
Benign	0.01	0.88	0.02	8
Malignant	1.00	0.26	0.41	1088

accuracy			0.27	1096
macro avg	0.50	0.57	0.22	1096
weighted avg	0.99	0.27	0.41	1096

Confusion matrix [[TN, FP], [FN, TP]]:

```
[[ 7  1]
 [804 284]]
```

Well we got most of the benigns—but at what cost? Kinda a wash

3.4.5 4.5: svm cascade idea on the synthetic set

```
[25]: cascade_syn = cascade_predict(X_test_syn, rf, svm_refiner, THRESHOLD)
print(f"=== CASCADE on TIER 1 (synthetic) ===")
print(f"Accuracy: {accuracy_score(y_test_syn, cascade_syn)*100:.2f}%")
print(classification_report(y_test_syn, cascade_syn,
                             target_names=['Benign', 'Malignant'],
                             zero_division=0))
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(confusion_matrix(y_test_syn, cascade_syn))
```

```
=== CASCADE on TIER 1 (synthetic) ===
```

```
Accuracy: 60.57%
```

	precision	recall	f1-score	support
Benign	0.56	0.95	0.71	1088
Malignant	0.84	0.26	0.40	1088
accuracy			0.61	2176
macro avg	0.70	0.61	0.55	2176
weighted avg	0.70	0.61	0.55	2176

Confusion matrix [[TN, FP], [FN, TP]]:

```
[[1034  54]
 [ 804 284]]
```

Why round 4 still doesn't “work” — and why that's the point

The tabular model achieves ~99% sensitivity at threshold 0.15 on synthetic benigns but 0% recall on the 8 held-out real benigns — every real case gets $P(\text{malignant}) > 0.74$. This isn't a bug. It illustrates that **demographics and anatomic site alone cannot reliably separate benign from malignant bone tumors**: the two classes overlap substantially on every feature SEER provides.

The strong synthetic-set performance reflects the model's ability to distinguish real SEER records from our epidemiology-based synthetic distribution — a related but distinct task from clinical diagnosis.

This motivates the hybrid approach. Tabular features provide weak demographic context; the image-based NN contributes the discriminative radiological features (matrix mineralization, pe-

riosteal reaction, cortical involvement) that radiologists actually use diagnostically.

4 hybrid models

using the improved MC sampling

4.1 1. shah's og transfer learning nn

Transfer learning from the mura_to_btprd_pipeline notebook

4.1.1 1.1 imports and paths

```
[26]: import cv2
from pathlib import Path
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.resnet_v2 import preprocess_input
from sklearn.ensemble import RandomForestClassifier

BTXRD_XLSX      = Path("data/BTXRD/dataset.xlsx")
BTXRD_IMAGES    = Path("data/BTXRD/images")
IMAGE_MODEL_PATH = Path("models/checkpoints/btxrd_from_mura.keras")

for p in (BTXRD_XLSX, BTXRD_IMAGES, IMAGE_MODEL_PATH):
    if not p.exists():
        raise FileNotFoundError(f"Missing required path: {p}")

image_model = load_model(IMAGE_MODEL_PATH, compile=False)
print(f"Loaded image model from: {IMAGE_MODEL_PATH}")
```

Loaded image model from: models\checkpoints\btxrd_from_mura.keras

4.1.2 1.2 load figshare (aka btxrd), define features, build paired dataframe

```
[27]: btxrd_df = pd.read_excel(BTXRD_XLSX, engine="openpyxl")
btxrd_df.columns = [str(c).strip() for c in btxrd_df.columns]

# Tabular feature columns. Everything is already numeric (age) or 0/1 dummies
# in the spreadsheet, so no encoding work needed.
TABULAR_FEATURES = [
    'age',
    # bone location flags
    'hand', 'ulna', 'radius', 'humerus', 'foot', 'tibia', 'fibula', 'femur',
    'hip bone',
    # joint flags
    'ankle-joint', 'knee-joint', 'hip-joint',
    'wrist-joint', 'elbow-joint', 'shoulder-joint',
    # body-region flags
    'upper limb', 'lower limb', 'pelvis',
```

```

# imaging view flags
'frontal', 'lateral', 'oblique',
]
LABEL_COL = 'malignant'    # 1 = malignant, 0 = not. Already in the spreadsheet.
IMAGE_COL = 'image_id'

# Encode gender as 0/1 (M=1, F=0)
btxrd_df['gender'] = (btxrd_df['gender'].astype(str).str.upper() == 'M').
    ↪astype(int)
TABULAR_FEATURES.insert(1, 'gender')

# Verify nothing is missing in the columns we need
required = TABULAR_FEATURES + [LABEL_COL, IMAGE_COL]
missing = [c for c in required if c not in btxrd_df.columns]
if missing:
    raise KeyError(f"BTXRD spreadsheet missing required columns: {missing}")

# Resolve image paths (same helper as before)
def resolve_image_path(images_dir, image_name):
    image_name = str(image_name).strip()
    direct = images_dir / image_name
    if direct.exists():
        return direct
    stem = Path(image_name).stem
    for ext in (".jpg", ".jpeg", ".png", ".JPG", ".JPEG", ".PNG"):
        candidate = images_dir / f"{stem}{ext}"
        if candidate.exists():
            return candidate
    matches = list(images_dir.rglob(f"{stem}.*"))
    return matches[0] if matches else None

btxrd_df['img_path'] = btxrd_df[IMAGE_COL].apply(
    lambda n: resolve_image_path(BTXRD_IMAGES, n))

# Drop any rows with missing data we need
before = len(btxrd_df)
btxrd_df = btxrd_df.dropna(subset=TABULAR_FEATURES + [LABEL_COL, 'img_path']).
    ↪reset_index(drop=True)
btxrd_df['img_path'] = btxrd_df['img_path'].astype(str)
btxrd_df[LABEL_COL] = btxrd_df[LABEL_COL].astype(int)
print(f"Paired patients: {len(btxrd_df)} (dropped {before - len(btxrd_df)} rows,
    ↪with missing data)")
print(f"Class balance: {btxrd_df[LABEL_COL].value_counts().to_dict()}")

```

Paired patients: 3746 (dropped 0 rows with missing data)

Class balance: {0: 3404, 1: 342}

unbalanced, but this is workabale

4.1.3 1.3 train/test split

```
[28]: # Stratified 80/20 split. Each row is one patient; img_path and tabular_
      ↪ features travel together so they stay paired through the split.
      from sklearn.model_selection import train_test_split

      train_btxrd, test_btxrd = train_test_split(
          btxrd_df, test_size=0.2, random_state=4337,
          stratify=btxrd_df[LABEL_COL]) #### stratified!

      print(f"Train: {len(train_btxrd)} patients, "
            f"class balance {train_btxrd[LABEL_COL].value_counts().to_dict()}")
      print(f"Test: {len(test_btxrd)} patients, "
            f"class balance {test_btxrd[LABEL_COL].value_counts().to_dict()}")
```

Train: 2996 patients, class balance {0: 2722, 1: 274}

Test: 750 patients, class balance {0: 682, 1: 68}

4.1.4 1.4 train the figshare tabular rf

```
[29]: X_train_btxrd = train_btxrd[TABULAR_FEATURES]
      y_train_btxrd = train_btxrd[LABEL_COL]
      X_test_btxrd  = test_btxrd[TABULAR_FEATURES]
      y_test_btxrd  = test_btxrd[LABEL_COL]

      rf_btxrd = RandomForestClassifier(
          n_estimators=200, random_state=4337, class_weight='balanced')
      rf_btxrd.fit(X_train_btxrd, y_train_btxrd)

      # Quick sanity check at default threshold
      from sklearn.metrics import confusion_matrix
      preds = rf_btxrd.predict(X_test_btxrd)
      cm = confusion_matrix(y_test_btxrd, preds, labels=[0, 1])
      print("BTXRD tabular RF, default threshold:")
      print(f"Confusion matrix [[TN, FP], [FN, TP]]:\n{cm}")
      print(f"  benigns caught: {cm[0,0]}/{cm[0].sum()} | FN: {cm[1,0]}")

      # Top features
      importances = (pd.Series(rf_btxrd.feature_importances_, index=TABULAR_FEATURES)
                      .sort_values(ascending=False))
      print("\nFeature importances:")
      print(importances.to_string())
```

BTXRD tabular RF, default threshold:

Confusion matrix [[TN, FP], [FN, TP]]:

[[618 64]

[33 35]]

benigns caught: 618/682 | FN: 33

Feature importances:

age	0.431650
femur	0.178505
humerus	0.085720
tibia	0.063489
fibula	0.051838
hip bone	0.038505
gender	0.031750
ulna	0.019463
upper limb	0.014013
frontal	0.013927
radius	0.013595
lower limb	0.012591
lateral	0.010293
pelvis	0.009993
foot	0.009077
oblique	0.006435
hand	0.005364
knee-joint	0.001614
shoulder-joint	0.001450
hip-joint	0.000597
elbow-joint	0.000072
ankle-joint	0.000059
wrist-joint	0.000000

note. The malignants are about evenly split in-between true and false. That's bad. We need to fix that ### 1.5 run img model on test set

```
[30]: def prepare_image(img_path, target_size=(224, 224)):
    img = cv2.imread(str(img_path))
    if img is None:
        raise ValueError(f"Image not found or unreadable: {img_path}")
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, target_size)
    img = np.expand_dims(img, axis=0)
    return preprocess_input(img.astype(np.float32))

def get_image_probs(image_paths):
    probs = []
    for i, path in enumerate(image_paths):
        out = image_model.predict(prepare_image(path), verbose=0)
        p_malign = float(out[0][0]) if out.shape[1] == 1 else float(out[0][1])
        probs.append(p_malign)
        if i % 200 == 0:
            print(f" processed {i}/{len(image_paths)}")
    return np.array(probs)
```

```
# Critical: test_btxrd preserves the row order. img_paths and tabular features
    ↪are now paired by patient.
img_probs_test = get_image_probs(test_btxrd['img_path'].values)
tab_probs_test = rf_btxrd.predict_proba(X_test_btxrd)[: , 1]
```

```
print(f"\nImage P(malig) - min/mean/max: "
      f"{img_probs_test.min():.3f} / {img_probs_test.mean():.3f} / "
      ↪f"{img_probs_test.max():.3f}")
print(f"Tabular P(malig) - min/mean/max: "
      f"{tab_probs_test.min():.3f} / {tab_probs_test.mean():.3f} / "
      ↪f"{tab_probs_test.max():.3f}")
```

```
processed 0/750
processed 200/750
processed 400/750
processed 600/750
```

```
Image P(malig) - min/mean/max: 0.000 / 0.524 / 1.000
Tabular P(malig) - min/mean/max: 0.000 / 0.122 / 0.995
```

4.1.5 1.6 threshold sweep

```
[31]: IMAGE_WEIGHT    = 0.70
      TABULAR_WEIGHT  = 0.30

hybrid_probs = IMAGE_WEIGHT * img_probs_test + TABULAR_WEIGHT * tab_probs_test

print("=" * 78)
print(f"HYBRID THRESHOLD SWEEP "
      f"(image_weight={IMAGE_WEIGHT}, tabular_weight={TABULAR_WEIGHT})")
print("=" * 78)
print(f"{'Threshold':<12}{'Malig Recall':<15}{'Malig Prec':<14}"
      f"{'Benign Recall':<15}{'FN':<6}{'FP':<6}")
print("-" * 78)
for thresh in np.arange(0.05, 0.95, 0.05):
    preds = (hybrid_probs >= thresh).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test_btxrd, preds, labels=[0, 1]).
    ↪ravel()
    malig_recall = tp / (tp + fn) if (tp + fn) else 0
    malig_prec   = tp / (tp + fp) if (tp + fp) else 0
    benign_recall = tn / (tn + fp) if (tn + fp) else 0
    print(f"{thresh:<12.2f}{malig_recall:<15.3f}{malig_prec:<14.3f}"
          f"{benign_recall:<15.3f}{fn:<6}{fp:<6}")
```

```
=====
HYBRID THRESHOLD SWEEP (image_weight=0.7, tabular_weight=0.3)
=====
```

```
Threshold    Malig Recall    Malig Prec    Benign Recall    FN    FP
```

0.05	1.000	0.160	0.477	0	357
0.10	1.000	0.164	0.493	0	346
0.15	1.000	0.167	0.503	0	339
0.20	1.000	0.169	0.510	0	334
0.25	1.000	0.170	0.512	0	333
0.30	1.000	0.173	0.522	0	326
0.35	1.000	0.173	0.523	0	325
0.40	1.000	0.174	0.528	0	322
0.45	1.000	0.175	0.531	0	320
0.50	0.985	0.174	0.534	1	318
0.55	0.985	0.176	0.540	1	314
0.60	0.971	0.175	0.543	2	312
0.65	0.971	0.177	0.550	2	307
0.70	0.897	0.258	0.743	7	175
0.75	0.647	0.349	0.880	24	82
0.80	0.574	0.368	0.902	29	67
0.85	0.515	0.376	0.915	33	58
0.90	0.324	0.355	0.941	46	40

4.1.6 1.7 eval at chosen threshold + compare against components

```
[32]: from sklearn.metrics import classification_report

THRESHOLD = 0.45 # adjust based on the sweep above

# Hybrid
preds_hybrid = (hybrid_probs >= THRESHOLD).astype(int)
cm_hybrid = confusion_matrix(y_test_btxrd, preds_hybrid, labels=[0, 1])

# Image-only at the same threshold
preds_image = (img_probs_test >= THRESHOLD).astype(int)
cm_image = confusion_matrix(y_test_btxrd, preds_image, labels=[0, 1])

# Tabular-only at the same threshold
preds_tab = (tab_probs_test >= THRESHOLD).astype(int)
cm_tab = confusion_matrix(y_test_btxrd, preds_tab, labels=[0, 1])

def summary(name, cm):
    tn, fp, fn, tp = cm.ravel()
    print(f"{name}:")
    print(f"  [[TN={tn}, FP={fp}], [FN={fn}, TP={tp}]]")
    print(f"  malig recall = {tp/(tp+fn):.3f} | benign recall = {tn/(tn+fp):.3f}")
    print(f"  FN = {fn} | FP = {fp}\n")

print(f"=== Comparison at threshold {THRESHOLD} ===\n")
```

```

summary("IMAGE ONLY",    cm_image)
summary("TABULAR ONLY",  cm_tab)
summary("HYBRID",        cm_hybrid)

print("--- Hybrid classification report ---")
print(classification_report(y_test_btxrd, preds_hybrid,
                           target_names=['Benign', 'Malignant'],
                           ↪zero_division=0))

```

=== Comparison at threshold 0.45 ===

IMAGE ONLY:

```

[[TN=356, FP=326], [FN=0, TP=68]]
malig recall = 1.000 | benign recall = 0.522
FN = 0 | FP = 326

```

TABULAR ONLY:

```

[[TN=614, FP=68], [FN=30, TP=38]]
malig recall = 0.559 | benign recall = 0.900
FN = 30 | FP = 68

```

HYBRID:

```

[[TN=362, FP=320], [FN=0, TP=68]]
malig recall = 1.000 | benign recall = 0.531
FN = 0 | FP = 320

```

--- Hybrid classification report ---

	precision	recall	f1-score	support
Benign	1.00	0.53	0.69	682
Malignant	0.18	1.00	0.30	68
accuracy			0.57	750
macro avg	0.59	0.77	0.50	750
weighted avg	0.93	0.57	0.66	750

Whats this mean? At 0.7, the hybrid is meaningfully different from image-only — it's trading malignant recall for benign specificity. Whether that's good depends on the clinical use case. For a screening tool where missing cancer is catastrophic, image-only at 0.7 (FN=1) is better than hybrid at 0.7 (FN=6). For a triage tool where you want to reduce unnecessary follow-ups, hybrid at 0.7 is better.

However, our focus here was minimizing FN. We can get to 0% FN at Threshold = 0.45, but the hybrid model is literally worse than the image model atp. A wash.

4.2 2. baseline cnn

4.2.1 2.1 implement helper function

(used for all three cnns - however, will not be restated)

```
[33]: def evaluate_hybrid(model_path, model_label,
                        test_btxrd, X_test_btxrd, y_test_btxrd,
                        rf_btxrd, image_weight=0.70, tabular_weight=0.30,
                        sweep_min=0.05, sweep_max=0.95, sweep_step=0.05):
    """
    Run a saved image model on the BTXRD test set, build a hybrid with the
    BTXRD tabular RF, and report the threshold sweep.

    Returns (img_probs, tab_probs, hybrid_probs) so the caller can pick a
    threshold and run the per-component comparison cell after.
    """
    print(f"=== Loading {model_label} from {model_path} ===")
    img_model = load_model(model_path, compile=False)

    # The baseline/improved/augmented models do their own /255 rescaling inside
    # the network (Rescaling layer), so we feed them raw 0-255 floats -
    # no preprocess_input call here.
    def prepare_raw(img_path, target_size=(224, 224)):
        img = cv2.imread(str(img_path))
        if img is None:
            raise ValueError(f"Image not readable: {img_path}")
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, target_size)
        return np.expand_dims(img.astype(np.float32), axis=0)

    img_probs = []
    paths = test_btxrd['img_path'].values
    for i, path in enumerate(paths):
        out = img_model.predict(prepare_raw(path), verbose=0)
        p_malig = float(out[0][0]) if out.shape[1] == 1 else float(out[0][1])
        img_probs.append(p_malig)
        if i % 200 == 0:
            print(f" processed {i}/{len(paths)}")
    img_probs = np.array(img_probs)
    tab_probs = rf_btxrd.predict_proba(X_test_btxrd)[: , 1]
    hybrid_probs = image_weight * img_probs + tabular_weight * tab_probs

    print(f"\nImage P(malig) - min/mean/max: "
          f"{img_probs.min():.3f} / {img_probs.mean():.3f} / {img_probs.max():.3f}")
    print(f"Tabular P(malig) - min/mean/max: "
```

```

        f"{tab_probs.min():.3f} / {tab_probs.mean():.3f} / {tab_probs.max():.3f}")

    print("\n" + "=" * 78)
    print(f"HYBRID THRESHOLD SWEEP - {model_label} "
          f"(image_weight={image_weight}, tabular_weight={tabular_weight})")
    print("=" * 78)
    print(f"{'Threshold':<12}{'Malig Recall':<15}{'Malig Prec':<14}"
          f"{'Benign Recall':<15}{'FN':<6}{'FP':<6}")
    print("-" * 78)
    for thresh in np.arange(sweep_min, sweep_max, sweep_step):
        preds = (hybrid_probs >= thresh).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_test_btxrd, preds, labels=[0, 1]).ravel()

        malig_recall = tp / (tp + fn) if (tp + fn) else 0
        malig_prec = tp / (tp + fp) if (tp + fp) else 0
        benign_recall = tn / (tn + fp) if (tn + fp) else 0
        print(f"{thresh:<12.2f}{malig_recall:<15.3f}{malig_prec:<14.3f}"
              f"{benign_recall:<15.3f}{fn:<6}{fp:<6}")

    return img_probs, tab_probs, hybrid_probs

def compare_at_threshold(img_probs, tab_probs, hybrid_probs, y_test, threshold,
                        model_label):
    """Side-by-side image / tabular / hybrid confusion matrices at one_
    threshold."""
    preds_img = (img_probs >= threshold).astype(int)
    preds_tab = (tab_probs >= threshold).astype(int)
    preds_hybrid = (hybrid_probs >= threshold).astype(int)

    def summary(name, preds):
        cm = confusion_matrix(y_test, preds, labels=[0, 1])
        tn, fp, fn, tp = cm.ravel()
        print(f"{name}:")
        print(f" [[TN={tn}, FP={fp}], [FN={fn}, TP={tp}]]")
        recall = tp/(tp+fn) if (tp+fn) else 0
        benign = tn/(tn+fp) if (tn+fp) else 0
        print(f" malig recall = {recall:.3f} | benign recall = {benign:.3f}")
        print(f" FN = {fn} | FP = {fp}\n")

    print(f"=== {model_label} comparison at threshold {threshold} ===\n")
    summary("IMAGE ONLY", preds_img)
    summary("TABULAR ONLY", preds_tab)
    summary("HYBRID", preds_hybrid)
    print(f"--- {model_label} hybrid classification report ---")
    print(classification_report(y_test, preds_hybrid,

```

```
target_names=['Benign', 'Malignant'],
zero_division=0))
```

4.2.2 2.2 finding threshold

```
[34]: baseline_path = Path("models/checkpoints/btxrd_baseline.keras")

img_probs_baseline, tab_probs_baseline, hybrid_probs_baseline = evaluate_hybrid(
    model_path=baseline_path,
    model_label="BASELINE CNN",
    test_btxrd=test_btxrd,
    X_test_btxrd=X_test_btxrd,
    y_test_btxrd=y_test_btxrd,
    rf_btxrd=rf_btxrd,
)
```

```
=== Loading BASELINE CNN from models\checkpoints\btxrd_baseline.keras ===
processed 0/750
processed 200/750
processed 400/750
processed 600/750
```

Image P(malig) - min/mean/max: 0.014 / 0.485 / 0.963

Tabular P(malig) - min/mean/max: 0.000 / 0.122 / 0.995

```
=====
HYBRID THRESHOLD SWEEP - BASELINE CNN (image_weight=0.7, tabular_weight=0.3)
=====
```

Threshold	Malig Recall	Malig Prec	Benign Recall	FN	FP
0.05	1.000	0.094	0.038	0	656
0.10	1.000	0.101	0.117	0	602
0.15	1.000	0.110	0.195	0	549
0.20	1.000	0.121	0.274	0	495
0.25	1.000	0.134	0.356	0	439
0.30	0.985	0.146	0.427	1	391
0.35	0.971	0.169	0.523	2	325
0.40	0.941	0.190	0.601	4	272
0.45	0.882	0.219	0.686	8	214
0.50	0.809	0.257	0.767	13	159
0.55	0.706	0.298	0.834	20	113
0.60	0.588	0.333	0.883	28	80
0.65	0.412	0.359	0.927	40	50
0.70	0.338	0.404	0.950	45	34
0.75	0.250	0.405	0.963	51	25
0.80	0.162	0.367	0.972	57	19
0.85	0.059	0.308	0.987	64	9
0.90	0.000	0.000	0.997	68	2

4.2.3 2.3.1 deets @ chosen threshold (.25)

```
[35]: THRESHOLD_BASELINE = 0.25
```

```
compare_at_threshold(  
    img_probs=img_probs_baseline,  
    tab_probs=tab_probs_baseline,  
    hybrid_probs=hybrid_probs_baseline,  
    y_test=y_test_btprd,  
    threshold=THRESHOLD_BASELINE,  
    model_label="BASELINE CNN",  
)
```

=== BASELINE CNN comparison at threshold 0.25 ===

IMAGE ONLY:

```
[[TN=162, FP=520], [FN=0, TP=68]]  
malig recall = 1.000 | benign recall = 0.238  
FN = 0 | FP = 520
```

TABULAR ONLY:

```
[[TN=599, FP=83], [FN=25, TP=43]]  
malig recall = 0.632 | benign recall = 0.878  
FN = 25 | FP = 83
```

HYBRID:

```
[[TN=243, FP=439], [FN=0, TP=68]]  
malig recall = 1.000 | benign recall = 0.356  
FN = 0 | FP = 439
```

--- BASELINE CNN hybrid classification report ---

	precision	recall	f1-score	support
Benign	1.00	0.36	0.53	682
Malignant	0.13	1.00	0.24	68
accuracy			0.41	750
macro avg	0.57	0.68	0.38	750
weighted avg	0.92	0.41	0.50	750

lousy performance ngl. But we knew this - Andy said so. Performed worse than transfer learning dataset

4.2.4 2.3.2 deets at chosen threshold (.35)

```
[48]: THRESHOLD_BASELINE = 0.35
```

```
compare_at_threshold(  
    img_probs=img_probs_baseline,  
    tab_probs=tab_probs_baseline,  
    hybrid_probs=hybrid_probs_baseline,  
    y_test=y_test_btxrd,  
    threshold=THRESHOLD_BASELINE,  
    model_label="BASELINE CNN",  
)
```

=== BASELINE CNN comparison at threshold 0.35 ===

IMAGE ONLY:

```
[[TN=243, FP=439], [FN=2, TP=66]]  
malig recall = 0.971 | benign recall = 0.356  
FN = 2 | FP = 439
```

TABULAR ONLY:

```
[[TN=609, FP=73], [FN=29, TP=39]]  
malig recall = 0.574 | benign recall = 0.893  
FN = 29 | FP = 73
```

HYBRID:

```
[[TN=357, FP=325], [FN=2, TP=66]]  
malig recall = 0.971 | benign recall = 0.523  
FN = 2 | FP = 325
```

--- BASELINE CNN hybrid classification report ---

	precision	recall	f1-score	support
Benign	0.99	0.52	0.69	682
Malignant	0.17	0.97	0.29	68
accuracy			0.56	750
macro avg	0.58	0.75	0.49	750
weighted avg	0.92	0.56	0.65	750

managed benign recall of over .5 while keeping malignant recall at .97. Not bad

4.2.5 2.3 TRYING CASCADE THING

```
[36]: def cascade_eval(img_probs, X_test, y_test, rf_model,  
    image_threshold, tabular_threshold, model_label):  
    """
```

```

Two-stage cascade:
    Stage 1: image flags a row malignant if img_prob >= image_threshold (wide_
↳net).
    Stage 2: tabular RF re-evaluates ONLY flagged rows. If RF's P(malig) is
        below tabular_threshold, downgrade to benign. Otherwise keep
        malignant.
    Anything not flagged by stage 1 stays benign - tabular never sees it.
    """
flagged = img_probs >= image_threshold
preds = np.zeros(len(img_probs), dtype=int)

if flagged.any():
    X_flagged = X_test.iloc[flagged]
    rf_probs_flagged = rf_model.predict_proba(X_flagged)[: , 1]
    # Keep as malignant only if RF also thinks it's malignant
    preds[flagged] = (rf_probs_flagged >= tabular_threshold).astype(int)

cm = confusion_matrix(y_test, preds, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()
print(f"=== {model_label} CASCADE: image@{image_threshold} ↳
↳tabular@{tabular_threshold} ===")
print(f" Stage 1 flagged {flagged.sum()}/{len(flagged)} cases as_
↳malignant")
print(f" Stage 2 downgraded {flagged.sum() - preds.sum()} of those to_
↳benign")
print(f" [[TN={tn}, FP={fp}], [FN={fn}, TP={tp}]]")
recall = tp/(tp+fn) if (tp+fn) else 0
benign = tn/(tn+fp) if (tn+fp) else 0
print(f" malig recall = {recall:.3f} | benign recall = {benign:.3f}")
print(f" FN = {fn} | FP = {fp}")
return preds, cm

```

```

[38]: # Sweep over image stage thresholds and tabular stage thresholds together
print(f"{'Img thresh':<12}{'Tab thresh':<12}{'FN':<6}{'FP':<6}{'TN':<6}{'TP':
↳<6}{'Notes'}")
print("-" * 70)

for img_t in [0.25, 0.30, 0.35, 0.40, 0.45]:
    for tab_t in [0.30, 0.40, 0.50, 0.60, 0.70]:
        flagged = img_probs_baseline >= img_t
        preds = np.zeros(len(img_probs_baseline), dtype=int)
        if flagged.any():
            X_flagged = X_test_btxrd.iloc[flagged]
            rf_probs_flagged = rf_btxrd.predict_proba(X_flagged)[: , 1]
            preds[flagged] = (rf_probs_flagged >= tab_t).astype(int)
            tn, fp, fn, tp = confusion_matrix(y_test_btxrd, preds, labels=[0,1]).
↳ravel()

```



```
print(f"{img_t:<12.2f}{tab_t:<12.2f}{fn:<6}{fp:<6}{tn:<6}{tp:<6}")
```

Img thresh	Tab thresh	FN	FP	TN	TP	Notes
0.25	0.30	28	76	606	40	
0.25	0.40	29	69	613	39	
0.25	0.50	33	62	620	35	
0.25	0.60	41	52	630	27	
0.25	0.70	50	36	646	18	
0.30	0.30	28	75	607	40	
0.30	0.40	29	68	614	39	
0.30	0.50	33	61	621	35	
0.30	0.60	41	51	631	27	
0.30	0.70	50	35	647	18	
0.35	0.30	28	72	610	40	
0.35	0.40	29	66	616	39	
0.35	0.50	33	60	622	35	
0.35	0.60	41	50	632	27	
0.35	0.70	50	34	648	18	
0.40	0.30	28	68	614	40	
0.40	0.40	29	62	620	39	
0.40	0.50	33	57	625	35	
0.40	0.60	41	49	633	27	
0.40	0.70	50	34	648	18	
0.45	0.30	30	66	616	38	
0.45	0.40	31	60	622	37	
0.45	0.50	35	55	627	33	
0.45	0.60	43	47	635	25	
0.45	0.70	51	32	650	17	

A flop. Nvm. Proceeding as normal now

4.3 3. improved cnn

4.3.1 3.1 finding threshold

```
[42]: # Improved
img_probs_improved, tab_probs_improved, hybrid_probs_improved = evaluate_hybrid(
    model_path=Path("models/checkpoints/btxrd_improved.keras"),
    model_label="IMPROVED CNN",
    test_btxrd=test_btxrd, X_test_btxrd=X_test_btxrd, y_test_btxrd=y_test_btxrd,
    rf_btxrd=rf_btxrd,
)
```

```
=== Loading IMPROVED CNN from models\checkpoints\btxrd_improved.keras ===
processed 0/750
processed 200/750
processed 400/750
processed 600/750
```

Image P(malig) - min/mean/max: 0.010 / 0.549 / 0.994
 Tabular P(malig) - min/mean/max: 0.000 / 0.122 / 0.995

```
=====
HYBRID THRESHOLD SWEEP - IMPROVED CNN (image_weight=0.7, tabular_weight=0.3)
=====
```

Threshold	Malig Recall	Malig Prec	Benign Recall	FN	FP
0.05	1.000	0.098	0.078	0	629
0.10	0.985	0.106	0.167	1	568
0.15	0.971	0.114	0.248	2	513
0.20	0.971	0.124	0.314	2	468
0.25	0.971	0.133	0.370	2	430
0.30	0.971	0.145	0.430	2	389
0.35	0.971	0.158	0.484	2	352
0.40	0.941	0.166	0.529	4	321
0.45	0.941	0.180	0.572	4	292
0.50	0.912	0.191	0.614	6	263
0.55	0.897	0.205	0.654	7	236
0.60	0.794	0.213	0.707	14	200
0.65	0.647	0.227	0.780	24	150
0.70	0.500	0.327	0.897	34	70
0.75	0.426	0.387	0.933	39	46
0.80	0.294	0.333	0.941	48	40
0.85	0.147	0.256	0.957	58	29
0.90	0.103	0.318	0.978	61	15

4.3.2 3.2.1 deets @ chosen threshold (.35)

```
[43]: THRESHOLD_IMPROVED = 0.35

compare_at_threshold(
    img_probs=img_probs_improved,
    tab_probs=tab_probs_improved,
    hybrid_probs=hybrid_probs_improved,
    y_test=y_test_btxrd,
    threshold=THRESHOLD_IMPROVED,
    model_label="IMPROVED CNN",
)

=== IMPROVED CNN comparison at threshold 0.35 ===

IMAGE ONLY:
[[TN=254, FP=428], [FN=3, TP=65]]
malig recall = 0.956 | benign recall = 0.372
FN = 3 | FP = 428
```

TABULAR ONLY:

```
[[TN=609, FP=73], [FN=29, TP=39]]
malig recall = 0.574 | benign recall = 0.893
FN = 29 | FP = 73
```

HYBRID:

```
[[TN=330, FP=352], [FN=2, TP=66]]
malig recall = 0.971 | benign recall = 0.484
FN = 2 | FP = 352
```

--- IMPROVED CNN hybrid classification report ---

	precision	recall	f1-score	support
Benign	0.99	0.48	0.65	682
Malignant	0.16	0.97	0.27	68
accuracy			0.53	750
macro avg	0.58	0.73	0.46	750
weighted avg	0.92	0.53	0.62	750

This sucks less. Higher benign recall (.48 vs .36) and only marginally worse FNs (.97 recall vs 1 - worthy tradeoff? unsure)

4.3.3 3.2.1 deets @ worse threshold (.05)

[44]: THRESHOLD_IMPROVED = 0.05

```
compare_at_threshold(
    img_probs=img_probs_improved,
    tab_probs=tab_probs_improved,
    hybrid_probs=hybrid_probs_improved,
    y_test=y_test_btxrd,
    threshold=THRESHOLD_IMPROVED,
    model_label="IMPROVED CNN",
)
```

=== IMPROVED CNN comparison at threshold 0.05 ===

IMAGE ONLY:

```
[[TN=33, FP=649], [FN=0, TP=68]]
malig recall = 1.000 | benign recall = 0.048
FN = 0 | FP = 649
```

TABULAR ONLY:

```
[[TN=557, FP=125], [FN=18, TP=50]]
malig recall = 0.735 | benign recall = 0.817
FN = 18 | FP = 125
```

HYBRID:

```
[[TN=53, FP=629], [FN=0, TP=68]]
malig recall = 1.000 | benign recall = 0.078
FN = 0 | FP = 629
```

--- IMPROVED CNN hybrid classification report ---

	precision	recall	f1-score	support
Benign	1.00	0.08	0.14	682
Malignant	0.10	1.00	0.18	68
accuracy			0.16	750
macro avg	0.55	0.54	0.16	750
weighted avg	0.92	0.16	0.15	750

However if we go balls to the wall and make sure $FN = 0$, it's worse, by a longshot. Benign recall of .08? Cmon

4.4 4. augmented cnn

4.4.1 4.1 finding threshold

```
[45]: img_probs_augmented, tab_probs_augmented, hybrid_probs_augmented = evaluate_hybrid(
    model_path=Path("models/checkpoints/btxrd_augmented.keras"),
    model_label="AUGMENTED CNN",
    test_btxrd=test_btxrd,
    X_test_btxrd=X_test_btxrd,
    y_test_btxrd=y_test_btxrd,
    rf_btxrd=rf_btxrd,
)
```

```
=== Loading AUGMENTED CNN from models\checkpoints\btxrd_augmented.keras ===
processed 0/750
processed 200/750
processed 400/750
processed 600/750
```

```
Image P(malig) - min/mean/max: 0.012 / 0.435 / 0.967
Tabular P(malig) - min/mean/max: 0.000 / 0.122 / 0.995
```

=====

HYBRID THRESHOLD SWEEP - AUGMENTED CNN (image_weight=0.7, tabular_weight=0.3)

=====

Threshold	Malig Recall	Malig Prec	Benign Recall	FN	FP
0.05	1.000	0.093	0.032	0	660

0.10	0.971	0.103	0.158	2	574
0.15	0.926	0.113	0.273	5	496
0.20	0.912	0.126	0.368	6	431
0.25	0.868	0.135	0.447	9	377
0.30	0.853	0.149	0.515	10	331
0.35	0.750	0.155	0.592	17	278
0.40	0.647	0.157	0.652	24	237
0.45	0.574	0.170	0.720	29	191
0.50	0.544	0.190	0.768	31	158
0.55	0.485	0.202	0.809	35	130
0.60	0.382	0.220	0.865	42	92
0.65	0.294	0.290	0.928	48	49
0.70	0.191	0.325	0.960	55	27
0.75	0.147	0.323	0.969	58	21
0.80	0.074	0.250	0.978	63	15
0.85	0.044	0.250	0.987	65	9
0.90	0.000	0.000	0.994	68	4

4.4.2 4.2 deerts @ chosen threshold

```
[47]: THRESHOLD_AUGMENTED = 0.20
```

```
compare_at_threshold(
    img_probs=img_probs_augmented,
    tab_probs=tab_probs_augmented,
    hybrid_probs=hybrid_probs_augmented,
    y_test=y_test_btxrd,
    threshold=THRESHOLD_AUGMENTED,
    model_label="AUGMENTED CNN",
)
```

=== AUGMENTED CNN comparison at threshold 0.2 ===

IMAGE ONLY:

```
[[TN=187, FP=495], [FN=9, TP=59]]
malig recall = 0.868 | benign recall = 0.274
FN = 9 | FP = 495
```

TABULAR ONLY:

```
[[TN=596, FP=86], [FN=25, TP=43]]
malig recall = 0.632 | benign recall = 0.874
FN = 25 | FP = 86
```

HYBRID:

```
[[TN=251, FP=431], [FN=6, TP=62]]
malig recall = 0.912 | benign recall = 0.368
FN = 6 | FP = 431
```

```

--- AUGMENTED CNN hybrid classification report ---
      precision    recall  f1-score   support

   Benign         0.98      0.37      0.53      682
  Malignant        0.13      0.91      0.22       68

 accuracy                   0.42      750
 macro avg         0.55      0.64      0.38      750
weighted avg         0.90      0.42      0.51      750

```

This just sucks. Idk. The hybrid model so far was the baseline CNN at threshold = .35